

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-002
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: DISEÑO DE SOFTWARE / ARQUITECTURA	ELABORACIÓN: 28/08/2026
	DISEÑO DE ARQUITECTURA DE SOFTWARE, MODELO C4 Y CLEAN ARCHITECTURE	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

1. TÍTULO

Denominación del Entregable: Documento de Arquitectura de Software (SAD), Modelo C4 y Patrones de Diseño (EDT 2.2)

Ficha Técnica de Identificación:

- **Código Documental:** TI-SIGE-ENT-S2-002 — **Versión:** 1.0
- **Institución:** Universidad Técnica de Ambato (UTA) – FISEI
- **Carrera:** Ingeniería de Software (Séptimo Semestre .A'')
- **Módulo Académico:** Gestión de Proyectos de Software
- **Docente Tutor:** Ing. Paulo Torres
- **Fecha de Elaboración:** 28/08/2026 — **Estado:** **En Elaboración / Pendiente de Revisión**
- **Período de Ejecución:** Semana 2 (27/08/2026 al 29/08/2026)
- **Responsable:** Steeven Toala (Líder de Proyecto / Arquitecto de Software)

2. INTRODUCCIÓN

La arquitectura de software del sistema **SIGE** se concibe bajo principios de alta cohesión, bajo acoplamiento, escalabilidad e independencia de frameworks. Se implementa una **Arquitectura Limpia (Clean Architecture)** en capas concéntricas sobre **.NET 9**, complementada con el **Modelo C4** para la visualización estructural del sistema y especificaciones rigurosas de seguridad basadas en JSON Web Tokens (JWT) y Control de Acceso Basado en Roles (RBAC).

3. OBJETIVO

3.1. Objetivo General

Diseñar y formalizar la arquitectura de software del sistema SIGE mediante Clean Architecture y el Modelo C4, asegurando mantenibilidad, testabilidad y privacidad de datos.

3.2. Objetivos Específicos

1. Estructurar las 4 capas concéntricas de Clean Architecture (.NET 9 WebAPI).
2. Modelar el Diagrama de Contenedores C4 que articula Frontend, Backend, PostgreSQL 16, MinIO, LanguageTool y Google Gemini API.
3. Especificar los patrones de diseño aplicados (Repository, CQRS, Validation Gates, Adapter y Factory).

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-002
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: DISEÑO DE SOFTWARE / ARQUITECTURA	ELABORACIÓN: 28/08/2026
	DISEÑO DE ARQUITECTURA DE SOFTWARE, MODELO C4 Y CLEAN ARCHITECTURE	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

4. Formalizar los mecanismos de seguridad criptográfica del token JWT.

4. ALCANCE

4.1. Límites Arquitecturales (In-Scope)

- Estructura de capas de la solución .NET 9 (Domain, Application, Infrastructure, WebAPI).
- Comunicación entre el cliente SPA React 19 y los microservicios locales vía RESTful APIs.

4.2. Exclusiones (Out-of-Scope)

- Despliegue en clústeres multi-región o balanceadores de carga globales de pago.

5. DEFINICIONES

- **Clean Architecture:** Enfoque arquitectónico que prioriza la lógica de dominio sobre los frameworks y bases de datos.
- **Modelo C4:** Marco de trabajo para la diagramación de arquitectura en cuatro niveles jerárquicos (Contexto, Contenedores, Componentes, Código).
- **CQRS:** Patrón de segregación de responsabilidades de comandos (escritura) y consultas (lectura).
- **JWT (JSON Web Token):** Estándar abierto (RFC 7519) para transmitir información de identidad de forma segura.

6. RESPONSABILIDADES

Integrante	Rol en el Entregable	Responsabilidad Específica
Steeven Toala	Líder / Arquitecto	Diseño de capas, diagramación C4, especificación de patrones y seguridad JWT.
Nixon Hurtado	Backend / DBA	Aislamiento de la capa de persistencia e infraestructura EF Core.
Ing. Paulo Torres	Docente Tutor	Revisión y aprobación del documento de arquitectura de software.

Tabla 1: Matriz de Responsabilidades del Entregable EDT 2.2.

7. NORMATIVA LEGAL Y TÉCNICA

- **IEEE Std 1471-2000 / ISO/IEC 42010:2011:** Systems and software engineering – Architecture description.

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-002
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: DISEÑO DE SOFTWARE / ARQUITECTURA	ELABORACIÓN: 28/08/2026
	DISEÑO DE ARQUITECTURA DE SOFTWARE, MODELO C4 Y CLEAN ARCHITECTURE	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

- **OpenAPI Specification 3.0 (OAS):** Estándar para documentación de contratos REST.
- **RFC 7519:** JSON Web Token (JWT) Standard.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

8.1. Capas de Clean Architecture (.NET 9)

Para asegurar alta mantenibilidad, independencia de frameworks, bajo acoplamiento y testabilidad rigurosa, el backend de **SIGE** se estructura bajo el patrón **Clean Architecture (Arquitectura Limpia)** en .NET 9:

1. **Capa de Dominio (SIGE.Domain):** Entidades nucleares (Documento, ActividadMatriz, FirmaLegalizacion, EvidenciaMinio), Value Objects, Enums institucionales (EstadoDocumento, TipoInforme) y Domain Events.
2. **Capa de Aplicación (SIGE.Application):** Casos de uso estructurados mediante CQRS (Commands/Queries con MediatR), DTOs, contratos de interfaces y validadores deterministas con FluentValidation.
3. **Capa de Infraestructura (SIGE.Infrastructure):** DbContext de Entity Framework Core para PostgreSQL 16, repositorio cliente de MinIO S3, cliente HTTP para LanguageTool y motor QuestPDF / OpenXML.
4. **Capa de Presentación / WebAPI (SIGE.WebAPI):** Controladores RESTful, middlewares de autenticación JWT y RBAC, Swagger OpenAPI y configuración de arranque (Program.cs).

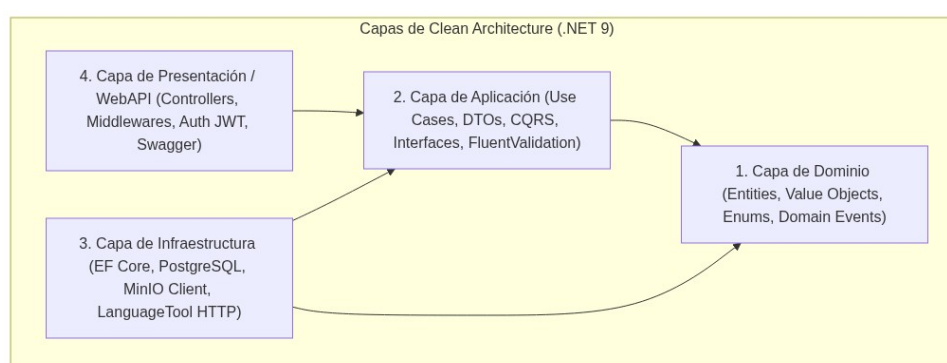


Figura 1: Capas Concéntricas de Clean Architecture en .NET 9.

8.2. Diagrama de Contenedores C4 de la Solución

El modelo C4 articula la interacción entre el cliente SPA y los microservicios locales On-Premise y en la nube:

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-002
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: DISEÑO DE SOFTWARE / ARQUITECTURA	ELABORACIÓN: 28/08/2026
	DISEÑO DE ARQUITECTURA DE SOFTWARE, MODELO C4 Y CLEAN ARCHITECTURE	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

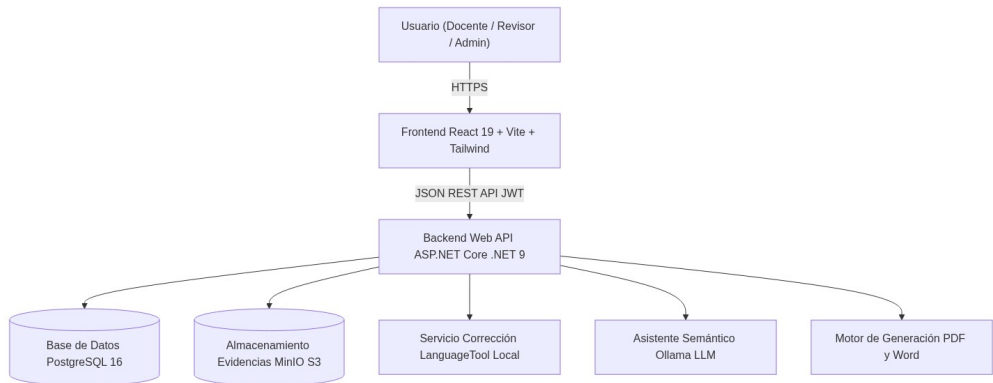


Figura 2: Diagrama de Contenedores del Modelo C4 para el Ecosistema SIGE.

- **Frontend SPA:** React 19 + TypeScript + Vite servido en Nginx (puertos 80/443).
- **Backend Web API:** ASP.NET Core .NET 9 (puerto 5000) autenticado con Bearer JWT.
- **Base de Datos Relacional:** PostgreSQL 16 (puerto 5432) con persistencia en volumen.
- **Almacenamiento de Evidencias:** MinIO S3 (puertos 9000 API y 9001 Consola Web).
- **Servicio de Corrección Sintáctica:** LanguageTool local en contenedor Docker (puerto 8010).
- **Asistente Semántico IA:** Inferencia delegada a Google Gemini API (Gemini 2.5 Flash / Google AI) mediante peticiones seguras HTTPS sin costos de suscripción ni sobrecarga de GPU local.
- **Motor de Documentos:** QuestPDF y OpenXML para renderizado fidedigno de PDF y Word .docx.

9. DESCRIPCIÓN DE ACTIVIDADES

9.1. Estructura de Proyectos en la Solución .NET 9

```
SIGE.Solution/
|-- src/
|   |-- SIGE.Domain/           # Entidades, Enums (EstadoDocumento,
|                               TipoInforme), Reglas
|   |-- SIGE.Application/      # Casos de uso (Commands/Queries),
|                               Validadores FluentValidation
|   |-- SIGE.Infrastructure/   # DbContext EF Core, MinIO S3 Repository,
|                               QuestPDF, LanguageTool API
|   |-- SIGE.WebAPI/           # Controladores REST, JWT Middleware,
|                               Swagger, Program.cs
|   '-- SIGE.ClientApp/        # Frontend React 19 + TypeScript + Vite +
|                               Tailwind CSS
```

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-002
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: DISEÑO DE SOFTWARE / ARQUITECTURA	ELABORACIÓN: 28/08/2026
	DISEÑO DE ARQUITECTURA DE SOFTWARE, MODELO C4 Y CLEAN ARCHITECTURE	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

```

'-- tests/
  |-- SIGE.UnitTests/          # Pruebas unitarias de Domain y Validation
    Gates (xUnit, Moq)
  '-- SIGE.IntegrationTests/# Pruebas de integracion de Endpoints
    WebAPI

```

9.2. Patrones de Diseño Implementados

1. **Repository & Unit of Work:** Desacopla la persistencia de datos y garantiza transacciones ACID en operaciones complejas (ej. cambio de estado + registro de auditoría + incremento de versión).
2. **CQRS con MediatR:** Separa las operaciones de lectura (*Queries*) de las operaciones de escritura/mutación (*Commands*).
3. **Pipeline Behaviors (Validation Gate):** Middleware de MediatR que intercepta cada *Command* y ejecuta automáticamente las reglas de FluentValidation antes de llegar a la capa de datos.
4. **Adapter Pattern para MinIO:** Interfaz *IObjectStorageService* que aísla la lógica del SDK de MinIO/S3.
5. **Factory Pattern para Exportación:** Generador dinámico que decide si renderizar en formato PDF (QuestPDF) o formato Word (OpenXML) según la solicitud institucional.

9.3. Especificación de Seguridad y Token JWT

- **Algoritmo de Firma Criptográfica:** HMAC-SHA256 con clave simétrica de 512 bits almacenada en variables de entorno seguras.
- **Tiempo de Expiración:** Access Token de corta duración (30 minutos) junto a un Refresh Token seguro de 7 días almacenado en base de datos con revocación automática.
- **Claims Oficiales en el Token:**
 - `sub (UserId)`: Identificador único universal del usuario (UUID).
 - `email`: Correo electrónico institucional (@uta.edu.ec).
 - `fullName`: Nombre completo del docente o directivo.
 - `role`: Rol RBAC (DocenteElaborador, CoordArea, CoordCarrera, Admin).
 - `facultadId` y `carreraId`: Pertenencia institucional a la FISEI e Ingeniería de Software.

	FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL	CÓDIGO: TI-SIGE-ENT-S2-002
	CARRERA DE INGENIERÍA DE SOFTWARE	VERSIÓN: 1.0
	PROCESO: DISEÑO DE SOFTWARE / ARQUITECTURA	ELABORACIÓN: 28/08/2026
	DISEÑO DE ARQUITECTURA DE SOFTWARE, MODELO C4 Y CLEAN ARCHITECTURE	ÚLTIMA REVISIÓN: PENDIENTE DE REVISIÓN

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Steeven Toala <i>Líder de Proyecto / Arquitecto Software</i>		28/08/2026
Revisado por:	Steeven Toala <i>Líder de Proyecto / Arquitecto Software</i>	[PENDIENTE DE REVISIÓN]	Pendiente
Aprobado por:	Ing. Paulo Torres <i>Docente Tutor - Gestión Proyectos Software</i>	[PENDIENTE DE APROBACIÓN]	Pendiente

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Diseño de arquitectura Clean Architecture en .NET 9, modelo C4 y especificación JWT / Pendiente de revisión	Steeven Toala	Pendiente de Revisión

12. ANEXOS

Anexo 1: Configuración de Dependencias en el Program.cs

Registro de servicios de inyección de dependencias para MediatR, DbContext, MinIO Client y Authentication JWT.